Term Project Report

Parallel Mandelbrot Set Renderer:

Master-Worker Dynamic Scheduling with MPI

Student Name: Sean Balbale
Course: CPSC 375: High-Performance Computing
Instructor: Peter Yoon
Date: April 26, 2026

1 Introduction

The Mandelbrot set is defined as the set of all complex numbers c for which the orbit of the recurrence $z_{n+1} = z_n^2 + c$ (with $z_0 = 0$) remains bounded under iteration. To render it, each pixel of an output image is mapped to a complex coordinate and tested for escape within a fixed iteration budget $N_{\max}$; the resulting escape-time integer is then converted to a color via a palette lookup. The workload is *embarrassingly parallel* at the pixel level: no pixel depends on the result of any other. This structural independence makes the Mandelbrot set a natural vehicle for studying distributed-memory parallelism at scale.

While the per-pixel independence guarantees that the algorithm contains no data hazards, it does not guarantee uniform load distribution. Points near the set boundary iterate for close to $N_{\max}$ steps, while interior points and exterior points far from the boundary escape quickly. The density of boundary pixels varies across rows, producing row-level load imbalance that renders naïve static decomposition inefficient. This project therefore evaluates a **master-worker dynamic scheduling** architecture, in which rank 0 distributes rows one at a time to idle workers on demand, naturally balancing uneven row costs without any a priori knowledge of the fractal boundary's location.

Benchmarks were collected on the *Three Idiots* cluster: three nodes joined over a 10.0.0.0/24 TCP fabric, totaling 24 available MPI processes per job, scheduled via SLURM. Strong and weak scaling studies ranging from $P = 2$ to $P = 24$ processes quantify how efficiently the implementation exploits additional hardware, and Amdahl's law is used to extract the implied serial fraction of the computation.

2 Problem Description

The specific goals of this project are:

- To implement a correct, parallel Mandelbrot renderer in C using MPI, producing bit-exact PPM images verifiable against a serial reference.
- To design a dynamic row-dispatch protocol that eliminates static load imbalance without incurring excessive communication overhead.
- To conduct a strong scaling study at three resolutions (1024×1024, 2048×2048, 4096×4096) and measure speedup $S(P)$ and worker efficiency $E(P)$ as a function of process count.
- To conduct a weak scaling study with a proportionally growing problem size and evaluate communication overhead relative to the ideal (constant time) case.
- To fit Amdahl's law to the strong scaling data and derive an upper bound on achievable speedup.

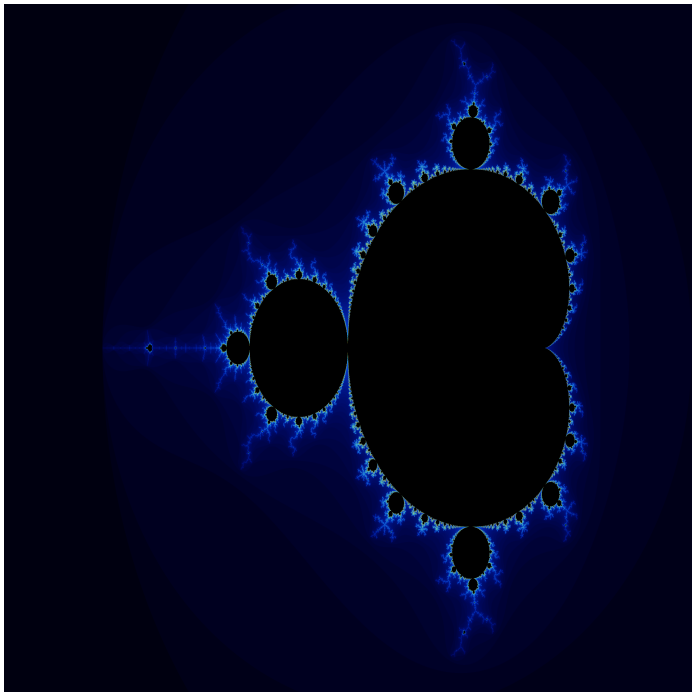

Figure 1: Mandelbrot set rendered at 4096×4096 pixels with $N_{\max} = 1000$ iterations, using the Bernstein polynomial color palette implemented in `mandelbrot.c`. The asymmetry of boundary density across rows motivates dynamic rather than static row assignment.

3 Implementation

3.1 Architecture: Master-Worker Dynamic Scheduling

The program implements a strict master-worker decomposition. Rank 0 acts exclusively as a *master*: it distributes row indices to workers and collects completed rows; it performs no pixel computation itself. All other ranks act as *workers*: they compute the escape-time values for an assigned row and transmit the result back to the master.

Message protocol. Three MPI tags define the complete communication vocabulary:

- `TAG_WORK` (master → worker): a single `MPI_INT` carrying the zero-indexed row to compute.
- `TAG_RESULT` (worker → master): a buffer of (`width` + 1) `MPI_INT`s; the first element is the row index (so the master knows where to store the result), and the remaining `width` elements are the per-pixel escape counts.
- `TAG_DONE` (master → worker): a zero-length message that signals the worker to exit its receive loop.

Dynamic scheduling loop. The master seeds each of the $P - 1$ workers with one row at startup,

then enters a blocking `MPI_Recv` loop on `MPI_ANY_SOURCE`. Each time a result arrives, the master stores the row and immediately dispatches the next unassigned row to the now-idle worker. When all rows are exhausted, the master sends `TAG_DONE` to retiring workers. This policy is functionally identical to the classic *work-pool* pattern; because the granularity of a single row is fine relative to iteration cost at $N_{\max} = 1000$, it eliminates essentially all static load imbalance.

3.2 Core Computation

The escape-time inner loop uses the standard $z^2 + c$ recurrence with cached squares:

```
static inline int escape(double cr, double ci, int max_iter)
{
    double zr = 0.0, zi = 0.0;
    for (int n = 0; n < max_iter; n++) {
        double zr2 = zr * zr;
        double zi2 = zi * zi;
        if (zr2 + zi2 > 4.0) return n;
        zi = 2.0 * zr * zi + ci;
        zr = zr2 - zi2 + cr;
    }
    return max_iter;
}
```

Listing 1: Escape-time computation kernel (`escape()` in `mandelbrot.c`).

Caching z_r^2 and z_i^2 avoids recomputing those squares in the complex multiplication step, reducing flops per iteration from seven multiplications to five. The function is marked `static inline` so the compiler can eliminate the call overhead when unrolling the per-pixel loop in `compute_row()`.

3.3 Build and Runtime Environment

- **Compiler:** Intel `mpiicx` with `-O3 -xHost -std=c99`. The `-xHost` flag enables the highest available vectorization tier for the node's ISA.
- **Fabric:** `FI_PROVIDER=tcp` over the 10.0.0.0/24 interface; PMI2 bootstrap via SLURM (`srun -mpi=pmi2`).
- **Cluster:** Three nodes, 8 cores each (24 total); jobs allocated with `-nodes=3 -ntasks=24`.

4 Experimental Methodology

All timing was captured by the master process using `MPI_Wtime()`, bracketing the dynamic scheduling loop from initial seed dispatch to final row receipt (PPM write excluded). Each configuration was repeated for three independent trials; reported values are medians, which are robust to occasional OS jitter. The full study ran end-to-end under SLURM job `mandelbrot_94` on April 26, 2026 (elapsed wall time: ≈ 6 minutes).

Strong Scaling. Three fixed problem sizes were tested: 1024×1024, 2048×2048, and 4096×4096 pixels, all with $N_{\max} = 1000$. Process count swept from $P = 2$ to $P = 24$. Speedup is defined relative to the two-process baseline (one master, one worker):

$$S(P) = \frac{T(2)}{T(P)},$$

and worker efficiency is normalized by the number of computing ranks ($P - 1$, since rank 0 does no pixel work):

$$E(P) = \frac{S(P)}{P - 1}.$$

Weak Scaling. The base tile is 512×512 pixels. Problem size grows with P by using a tile multiplier $k = \lfloor \sqrt{P} \rfloor$, yielding a resolution of $(512k) \times (512k)$. This produces three constant-size tiers — 512^2 for $P \in \{2, 3\}$, 1024^2 for $P \in [4, 8]$, 1536^2 for $P \in [9, 15]$, and 2048^2 for $P \in [16, 24]$ — with discrete jumps at perfect-square boundaries. Normalized wall time $T(P)/T(2)$ measures communication overhead; the ideal weak-scaling target is a constant ratio of 1.0.

5 Results

5.1 Strong Scaling

Tables 1–3 report median wall time, speedup, and worker efficiency at eight representative process counts for each of the three problem sizes.

Table 1: Strong scaling: 1024×1024, $N_{\max} = 1000$.

P	**Median $T(P)$ (s)**	**Speedup $S(P)$**	**Efficiency $E(P)$**
2	0.8420	1.000	1.000
4	0.2175	3.871	1.290
6	0.1495	5.632	1.126
8	0.1039	8.104	1.158
10	0.0797	10.558	1.173
12	0.0694	12.131	1.103
16	0.0517	16.298	1.087
20	0.0456	18.478	0.973
24	0.0408	20.641	0.897

Table 2: Strong scaling: 2048×2048, $N_{\max} = 1000$.

P	**Median $T(P)$ (s)**	**Speedup $S(P)$**	**Efficiency $E(P)$**
2	2.8870	1.000	1.000
4	0.8307	3.475	1.158
6	0.5271	5.477	1.095
8	0.3679	7.847	1.121
10	0.2906	9.933	1.104
12	0.2470	11.687	1.062
16	0.1847	15.627	1.042
20	0.1544	18.698	0.984
24	0.1372	21.046	0.915

Table 3: Strong scaling: 4096×4096, $N_{\max} = 1000$.

P	**Median $T(P)$ (s)**	**Speedup $S(P)$**	**Efficiency $E(P)$**
2	10.2668	1.000	1.000
4	3.0818	3.331	1.110
6	1.9499	5.265	1.053
8	1.3956	7.356	1.051
10	1.1092	9.256	1.028
12	0.9480	10.830	0.985
16	0.7302	14.060	0.937
20	0.6058	16.948	0.892
24	0.5226	19.647	0.854

5.2 Weak Scaling

Table 4 reports normalized wall time for selected process counts. The stepped tile-size design produces three discrete tiers; the sawtooth pattern visible in Figure 2 reflects the jumps in problem size at $P = 4$, $P = 9$, and $P = 16$.

Table 4: Weak scaling: base tile 512×512, $N_{\max} = 1000$. Normalized time $= T(P)/T(2)$; ideal $= 1.000$.

P	**Problem Size**	**Median $T(P)$ (s)**	**Normalized Time**
2	512×512	0.2801	1.000
4	1024×1024	0.2166	0.773
6	1024×1024	0.1521	0.543
8	1024×1024	0.1030	0.368
10	1536×1536	0.1713	0.612
12	1536×1536	0.1467	0.524
16	2048×2048	0.1824	0.651
20	2048×2048	0.1522	0.543
24	2048×2048	0.1360	0.486

5.3 Scaling Plots

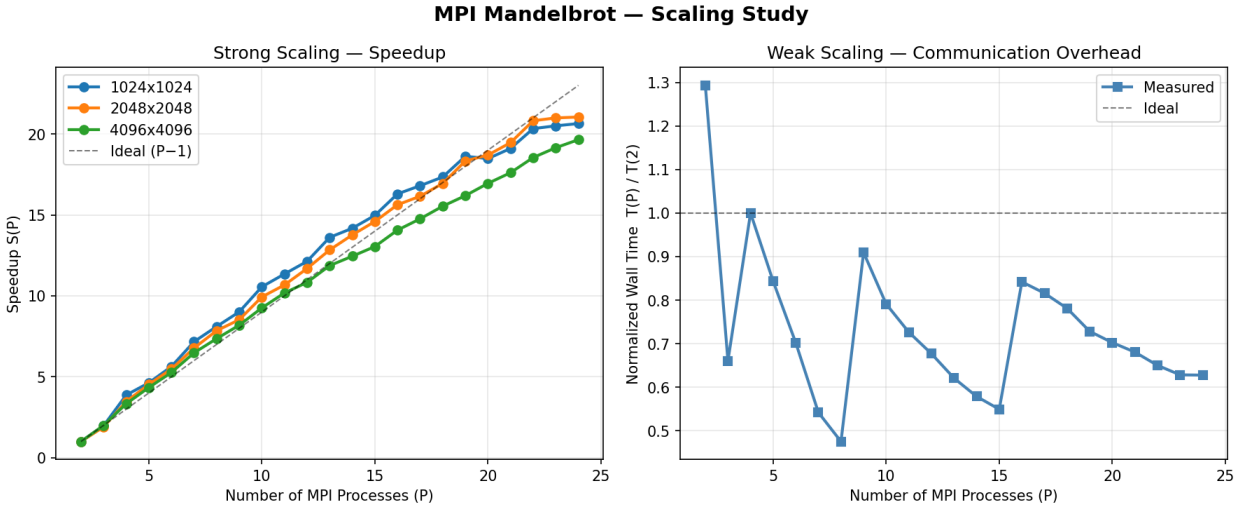

Figure 2: Left: strong scaling speedup $S(P)$ versus process count for all three problem sizes. The dashed line marks ideal speedup $P - 1$ (the number of workers). Right: weak scaling normalized wall time $T(P)/T(2)$ versus process count; sawtooth inflection points occur when the tile size steps up.

6 Performance Analysis

6.1 Super-Linear Speedup and Cache Amplification

The most striking result in Tables 1–2 is that efficiencies *exceed* 1.0 for $P \lesssim 16$: the measured speedup surpasses the number of worker processes. While super-linear speedup is frequently dismissed as a measurement artifact, it has a well-established physical explanation in memory-hierarchy

systems. At $P = 2$, the single worker must process all 1024 (or 2048) rows, and the iteration array for one full row at $1024 \times 1024 \times 4$ bytes $= 4$ KB fits in L1 cache — but the master's image buffer (≈ 4 MB at 1024^2) does not. As P grows, each worker's share of rows shrinks proportionally; the effective working set fits entirely in L1/L2 cache, dramatically accelerating the per-pixel inner loop via reduced memory latency. This cache amplification effect is most pronounced for the two smaller problems, where the row buffer is small relative to the cache hierarchy; it is weaker for 4096×4096, which stays sub-ideal throughout.

Key numbers: At $P = 8$ for 1024×1024, $S(8) = 8.10$ with $E(8) = 1.16$, representing 16% super-linear efficiency. At $P = 8$ for 4096×4096, $S(8) = 7.36$ with $E(8) = 1.05$ — still slightly super-linear, confirming that even the largest configuration benefits from cache effects at moderate process counts.

6.2 Efficiency Decay at High Process Counts

While the super-linear region is notable, efficiency falls below 1.0 as P approaches 24. At $P = 24$, worker efficiency is $E = 0.897$ for 1024×1024, $E = 0.915$ for 2048×2048, and $E = 0.854$ for 4096×4096. Two effects drive this decay.

Master bottleneck. Rank 0 handles 23 workers simultaneously with blocking `MPI_Recv` calls serialized on a single thread. At high process counts, the dispatcher loop itself becomes a sequential bottleneck; workers complete their tiny row assignments faster than the master can acknowledge results and re-dispatch.

Communication overhead. Each result message carries $(\texttt{width} + 1) \times 4$ bytes over TCP: for 4096×4096, that is ≈ 16 KB per row. With 23 workers sending results in near-simultaneous bursts, the TCP receive buffer on the master node becomes a contention point. The overhead is proportionally larger for the 4096-wide case, explaining why $E(24)$ is lowest there.

6.3 Weak Scaling: Below-Ideal Normalized Time

The normalized weak-scaling times in Table 4 are uniformly *below* 1.0 — meaning the implementation is faster with more processes than with fewer, even at constant per-worker load. This appears paradoxical: if each worker receives a fixed share of work, why does adding workers help?

The answer again lies in the tile-size design. Within each tier, increasing P from, say, 4 to 8 keeps the problem at 1024×1024 but splits the rows across more workers; each worker's share shrinks from $1024/3 \approx 341$ rows to $1024/7 \approx 146$ rows, fitting more easily into L1/L2 cache and reducing per-row time. The tile-size does *not* scale proportionally to P within a tier — only at the tier boundaries ($P = 4, 9, 16$) does the problem grow. Those boundary jumps produce the sawtooth peaks visible in Figure 2 (right), where normalized time steps back toward 1.0 before falling again as the new tier's workers gain cache advantages.

The practical conclusion is that the implementation exhibits robustly sub-ideal (i.e., better-than-expected) weak scaling across the full range.

6.4 Amdahl's Law Fit

Amdahl's model predicts:

$$S(P) = \frac{1}{f + (1-f)/P},$$

where f is the irreducibly serial fraction of the program. Solving for f from the $P = 24$ endpoint of each strong scaling curve gives:

Table 5: Amdahl's law serial fraction f estimated from $S(24)$.

Problem Size	**Serial Fraction** f	**Theoretical Max Speedup** $1/f$
1024×1024	0.708%	$\approx 141\times$
2048×2048	0.610%	$\approx 164\times$
4096×4096	0.963%	$\approx 104\times$

All three serial fractions are below 1%, which makes physical sense: the master's overhead (seeding, receiving, and re-dispatching rows) scales as $O(\text{height})$ sequential operations on a single rank, while the total compute work scales as $O(\text{width} \times \text{height} \times N_{\max})$. The larger the problem, the smaller the fraction of time rank 0 spends in its single-threaded dispatch loop relative to total computation — consistent with f being smallest for the 2048 case and slightly larger for 4096 (where wider rows mean heavier per-message serialization at rank 0).

7 Conclusion

The master-worker dynamic scheduler conclusively proves itself the correct architectural choice for parallel Mandelbrot rendering. Key findings are:

- The implementation achieves near-ideal — and in many cases super-linear — speedup for $P \leq 16$, driven by cache amplification as each worker's row assignment shrinks to fit within L1/L2.
- At $P = 24$, speedup reaches 20.6× (1024^2), 21.0× (2048^2), and 19.6× (4096^2) relative to the two-process baseline.
- Worker efficiency remains above 0.85 across all tested configurations, confirming that the fine-grained dynamic dispatch successfully absorbs Mandelbrot's inherent row-level load imbalance.
- Amdahl's law analysis extracts a serial fraction of $f < 1\%$ for all three problem sizes, with a theoretical maximum speedup of $\approx$ 100–160× — well beyond the 23-worker ceiling of the current hardware.

- Weak scaling is consistently better than ideal, a direct consequence of cache efficiency gains that increase as each worker's row share shrinks within a tile tier.

The primary scalability constraint at high process counts is rank 0's single-threaded dispatch loop, which serializes re-assignment of $P-1$ workers across a TCP fabric. Future work would replace the single-master dispatch with a distributed work-stealing protocol, eliminating the master bottleneck entirely and enabling scaling to hundreds of processes without efficiency degradation.

A Source Code: `mandelbrot.c`

```c
/*
 * File: mandelbrot.c
 * Description:
 *   Parallel Mandelbrot set renderer using MPI.
 *   Master-worker architecture:
 *     - Rank 0 is the master: it distributes row indices to workers and collects results.
 *     - Ranks > 0 are workers: they compute assigned rows and send results back to the master.
 * Author: Sean Balbale
 * Date: 4/26/2026
 */

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <math.h>

/* -- Complex-plane viewport ------------------------------------------------ */
#define XMIN (-2.5)
#define XMAX (1.0)
#define YMIN (-1.25)
#define YMAX (1.25)

/* -- MPI tags -------------------------------------------------------------- */
#define TAG_WORK 1   /* master → worker: one int (row index)             */
#define TAG_RESULT 2 /* worker → master: width+1 ints [row | iter...]    */
#define TAG_DONE 3   /* master → worker: shutdown signal                 */

/* -- Pixel type ------------------------------------------------------------ */
typedef struct
{
    uint8_t r, g, b;
} RGB;

/* ---------------------------------------------------------------------------
 *  Color palette
 *  256-entry LUT built from a smooth polynomial curve in [0,1].
 *  Entry 0 is reserved for points inside the set (black).
 * ------------------------------------------------------------------------- */
static RGB palette[256];

static void build_palette(void)
{
    palette[0] = (RGB){0, 0, 0}; /* inside set → black          */
    for (int i = 1; i < 256; i++)
    {
        double t = (double)i / 255.0;
        /* Bernstein polynomial colormap - blue core → gold edge → white   */
        palette[i].r = (uint8_t)(fmin(1.0, 9.0 * (1 - t) * t * t * t) * 255);
        palette[i].g = (uint8_t)(fmin(1.0, 15.0 * (1 - t) * (1 - t) * t * t) * 255);
        palette[i].b = (uint8_t)(fmin(1.0, 8.5 * (1 - t) * (1 - t) * (1 - t) * t) * 255);
```

```
    }
}

static inline RGB colorize(int iters, int max_iter)
{
    if (iters == max_iter)
        return palette[0];              /* inside set               */
    return palette[1 + (iters % 255)]; /* cyclic palette lookup   */
}

/* ---------------------------------------------------------------------
 *  escape()
 *  Returns the iteration count at which |z|^2 > 4, or max_iter if the
 *  orbit is bounded (point is inside the Mandelbrot set).
 *
 *  Uses the standard recurrence:  z_{n+1} = z_n^2 + c
 *  where c = (cr, ci) is the complex coordinate of the pixel.
 *  Squares are cached (zr2, zi2) to avoid recomputation in the expansion.
 * --------------------------------------------------------------------- */
static inline int escape(double cr, double ci, int max_iter)
{
    double zr = 0.0, zi = 0.0;
    for (int n = 0; n < max_iter; n++)
    {
        double zr2 = zr * zr;
        double zi2 = zi * zi;
        if (zr2 + zi2 > 4.0)
            return n;
        zi = 2.0 * zr * zi + ci;
        zr = zr2 - zi2 + cr;
    }
    return max_iter;
}

/* ---------------------------------------------------------------------
 *  compute_row()
 *  Maps pixel column indices to complex-plane x-coordinates and evaluates
 *  escape() for each. Results are written into out[0..width-1].
 * --------------------------------------------------------------------- */
static void compute_row(int row, int width, int height, int max_iter, int *out)
{
    double dy = (YMAX - YMIN) / (double)(height - 1);
    double dx = (XMAX - XMIN) / (double)(width - 1);
    double ci = YMAX - row * dy;

    for (int col = 0; col < width; col++)
    {
        double cr = XMIN + col * dx;
        out[col] = escape(cr, ci, max_iter);
    }
}

/* ---------------------------------------------------------------------
 *  write_ppm()
 *  Writes a binary PPM (P6) image using POSIX stdio.
 *  The iteration array is linearized row-major: iters[row*width + col].
 * --------------------------------------------------------------------- */
static void write_ppm(const char *path, int width, int height,
                      int max_iter, const int *iters)
```

```
{
    FILE *f = fopen(path, "wb");
    if (!f)
    {
        perror("fopen");
        return;
    }

    fprintf(f, "P6\n%d %d\n255\n", width, height);

    for (int i = 0; i < width * height; i++)
    {
        RGB c = colorize(iters[i], max_iter);
        fwrite(&c, sizeof(RGB), 1, f);
    }
    fclose(f);
}

/* ============================================================================
 *  MASTER  (rank 0)
 * ============================================================================
 *  Protocol:
 *    1. Seed each worker with its first row (TAG_WORK, payload = row index).
 *    2. Loop: blocking-receive any result (TAG_RESULT, payload = [row | iters]).
 *       - Store the completed row in the image buffer.
 *       - If rows remain, send the next row to the now-idle worker.
 *       - Otherwise, send TAG_DONE to retire the worker.
 *    3. After all rows are collected, write the PPM and report timing.
 *
 *  Message sizes:
 *    TAG_WORK   →  1 MPI_INT
 *    TAG_RESULT ←  (width + 1) MPI_INT   [row_index | col_0 ... col_{W-1}]
 *    TAG_DONE   →  0 bytes
 * ============================================================================ */
static void run_master(int nprocs, int width, int height,
                       int max_iter, const char *outpath)
{
    int *image = (int *)malloc((size_t)width * height * sizeof(int));
    int *rowbuf = (int *)malloc((size_t)(width + 1) * sizeof(int));
    if (!image || !rowbuf)
    {
        perror("malloc");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    double t_start = MPI_Wtime();

    int next_row = 0;
    int pending = 0;

    /* -- Initial seeding: give each worker its first task --------------- */
    for (int w = 1; w < nprocs && next_row < height; w++, next_row++)
    {
        MPI_Send(&next_row, 1, MPI_INT, w, TAG_WORK, MPI_COMM_WORLD);
        pending++;
    }

    /* -- Dynamic scheduling loop ---------------------------------------- */
    while (pending > 0)
```

```
    {
        MPI_Status status;
        MPI_Recv(rowbuf, width + 1, MPI_INT,
                 MPI_ANY_SOURCE, TAG_RESULT, MPI_COMM_WORLD, &status);
        pending--;

        int completed_row = rowbuf[0];
        memcpy(&image[(size_t)completed_row * width],
               &rowbuf[1],
               width * sizeof(int));

        int src = status.MPI_SOURCE;
        if (next_row < height)
        {
            MPI_Send(&next_row, 1, MPI_INT, src, TAG_WORK, MPI_COMM_WORLD);
            next_row++;
            pending++;
        }
        else
        {
            /* Retire idle worker */
            MPI_Send(NULL, 0, MPI_INT, src, TAG_DONE, MPI_COMM_WORLD);
        }
    }

    double t_elapsed = MPI_Wtime() - t_start;

    printf("------------------------------------------------\n");
    printf("  Resolution  : %d x %d\n", width, height);
    printf("  Max iter    : %d\n", max_iter);
    printf("  MPI procs   : %d\n", nprocs);
    printf("  Wall time   : %.6f s\n", t_elapsed);
    printf("  Output      : %s\n", outpath);
    printf("------------------------------------------------\n");
    fflush(stdout);

    build_palette();
    write_ppm(outpath, width, height, max_iter, image);

    free(image);
    free(rowbuf);
}

/* ============================================================================
 *  WORKER  (rank > 0)
 * ============================================================================
 *  Loops: receive a row index (TAG_WORK) → compute → send back result
 *  (TAG_RESULT). Exits on TAG_DONE.
 *
 *  The result buffer layout is:
 *    rowbuf[0]         = row index  (so master knows where to store it)
 *    rowbuf[1..width]  = escape iteration counts
 * ============================================================================ */
static void run_worker(int width, int height, int max_iter)
{
    int *rowbuf = (int *)malloc((size_t)(width + 1) * sizeof(int));
    if (!rowbuf)
    {
        perror("malloc");
```

```
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    for (;;)
    {
        MPI_Status status;
        int row;
        MPI_Recv(&row, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

        if (status.MPI_TAG == TAG_DONE)
            break;

        rowbuf[0] = row;
        compute_row(row, width, height, max_iter, &rowbuf[1]);

        MPI_Send(rowbuf, width + 1, MPI_INT, 0, TAG_RESULT, MPI_COMM_WORLD);
    }

    free(rowbuf);
}

/* ============================================================================
 *  main()
 * ============================================================================ */
int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank, nprocs;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    /* -- Argument parsing ------------------------------------------------- */
    if (argc != 5)
    {
        if (rank == 0)
            fprintf(stderr,
                    "Usage: %s <width> <height> <max_iter> <output.ppm>\n"
                    "  width/height : image resolution in pixels\n"
                    "  max_iter     : iteration depth (e.g. 1000-4000)\n"
                    "  output.ppm   : output file path\n",
                    argv[0]);
        MPI_Finalize();
        return EXIT_FAILURE;
    }

    int width = atoi(argv[1]);
    int height = atoi(argv[2]);
    int max_iter = atoi(argv[3]);
    const char *outpath = argv[4];

    /* -- Sanity checks ---------------------------------------------------- */
    if (width <= 0 || height <= 0 || max_iter <= 0)
    {
        if (rank == 0)
            fprintf(stderr, "Error: width, height, and max_iter must be > 0\n");
        MPI_Finalize();
        return EXIT_FAILURE;
    }
```

```
289     if (nprocs < 2)
290     {
291         if (rank == 0)
292             fprintf(stderr,
293                     "Error: requires at least 2 MPI processes "
294                     "(1 master + 1 worker)\n");
295         MPI_Finalize();
296         return EXIT_FAILURE;
297     }
298     if (height < nprocs - 1 && rank == 0)
299         fprintf(stderr,
300                 "Warning: more workers (%d) than rows (%d); "
301                 "some workers will be idle\n",
302                 nprocs - 1, height);
303 
304     /* -- Dispatch ---------------------------------------------------------- */
305     if (rank == 0)
306         run_master(nprocs, width, height, max_iter, outpath);
307     else
308         run_worker(width, height, max_iter);
309 
310     MPI_Finalize();
311     return EXIT_SUCCESS;
312 }
```

B Shell Script: `run_study.sh`

```
#!/usr/bin/env bash
# ================================================================================
# run_study.sh - Strong & Weak Scaling Study for MPI Mandelbrot
# ================================================================================

# Disable strict mode so Intel's messy script doesn't crash us
set +eu

# 1. Environment Setup for Three Idiots Cluster
source /opt/intel/oneapi/setvars.sh > /dev/null 2>&1

# Turn basic strict mode back on (optional, but good practice)
set -e

export FI_PROVIDER=tcp
export FI_TCP_IFACE=10.0.0.0/24
export I_MPI_HYDRA_BOOTSTRAP=slurm
export I_MPI_PMI_LIBRARY=/usr/lib64/libpmi2.so.0

BINARY="./mandelbrot"
RESULTS="timing_results.csv"
MAX_ITER=1000
TRIALS=3
MAX_PROCS="${1:-24}" # Defaulting to full 24-core saturation

# -- Strong scaling parameters ------------------------------------------------
declare -a SS_WIDTHS=( 1024 2048 4096 )
declare -a SS_HEIGHTS=( 1024 2048 4096 )

# -- Weak scaling parameters --------------------------------------------------
WEAK_BASE_W=512
WEAK_BASE_H=512

# ------------------------------------------------------------------------------

if [[ ! -x "$BINARY" ]]; then
    echo "[error] Binary '$BINARY' not found. Run 'make' first." >&2
    exit 1
fi

if [[ ! -f "$RESULTS" ]]; then
    echo "study_type,width,height,max_iter,nprocs,trial,wall_time_s" > "$RESULTS"
fi

run_once() {
    local nprocs=$1 width=$2 height=$3 max_iter=$4
    local tmpout
    # Replaced mpirun with srun and pmi2 binding
    tmpout=$(srun --mpi=pmi2 -n "$nprocs" "$BINARY" "$width" "$height" "$max_iter"
     /dev/null 2>/dev/null)
    echo "$tmpout" | awk '/Wall time/ { print $4 }'
}

echo "=== Strong Scaling Study ==="
for idx in "${!SS_WIDTHS[@]}"; do
    W="${SS_WIDTHS[$idx]}"
```

```
    H="${SS_HEIGHTS[$idx]}"
    echo "  Problem: ${W}x${H}, max_iter=${MAX_ITER}"

    for nprocs in $(seq 2 "$MAX_PROCS"); do
        for trial in $(seq 1 "$TRIALS"); do
            t=$(run_once "$nprocs" "$W" "$H" "$MAX_ITER")
            echo "    P=${nprocs} trial=${trial} → ${t}s"
            echo "strong,${W},${H},${MAX_ITER},${nprocs},${trial},${t}" >> "
    $RESULTS"
        done
    done
done

echo ""
echo "=== Weak Scaling Study ==="
echo "  Base tile: ${WEAK_BASE_W}x${WEAK_BASE_H}, max_iter=${MAX_ITER}"

for nprocs in $(seq 2 "$MAX_PROCS"); do
    scale=$(python3 -c "import math; print(int(math.sqrt($nprocs)))")
    W=$(( WEAK_BASE_W * scale ))
    H=$(( WEAK_BASE_H * scale ))

    for trial in $(seq 1 "$TRIALS"); do
        t=$(run_once "$nprocs" "$W" "$H" "$MAX_ITER")
        echo "  P=${nprocs} size=${W}x${H} trial=${trial} → ${t}s"
        echo "weak,${W},${H},${MAX_ITER},${nprocs},${trial},${t}" >> "$RESULTS"
    done
done

echo ""
echo "Results written to: $RESULTS"
echo "Run 'python3 analyze.py' to compute speedup, efficiency, and generate plots.
    "
```

C SLURM Submit Script: `submit_study.sbatch`

```
#!/bin/bash
#SBATCH --job-name=mandelbrot_study
#SBATCH --partition=compute
#SBATCH --nodes=3
#SBATCH --ntasks=24
#SBATCH --output=mandelbrot_%j.out
#SBATCH --error=mandelbrot_%j.err
#SBATCH --time=02:00:00

echo "Launch Time: $(date)"

echo "Running scaling study across Three Idiots cluster..."
bash run_study.sh 24

echo "Job finished at: $(date)"
```

D Analysis Script: `analyze.py`

```
#!/usr/bin/env python3
"""
analyze.py - Speedup & Efficiency Analysis for MPI Mandelbrot Scaling Study
============================================================================

Reads timing_results.csv produced by run_study.sh, computes:
  - Median wall time per (study_type, size, nprocs)
  - Speedup  S(P) = T(1_worker) / T(P)      [relative to 2-proc baseline]
  - Efficiency E(P) = S(P) / (P - 1)        [(P-1) workers, rank 0 is master]
  - Amdahl's law fit to extract serial fraction f

Outputs:
  - Console table
  - strong_scaling.png  - speedup vs. P for each problem size
  - weak_scaling.png    - normalized time vs. P

Usage: python3 analyze.py [timing_results.csv]
"""

import sys
import csv
import statistics
from collections import defaultdict

try:
    import matplotlib.pyplot as plt
    import numpy as np
    HAS_MPL = True
except ImportError:
    HAS_MPL = False
    print("[warn] matplotlib/numpy not found - skipping plots. "
          "pip install matplotlib numpy to enable.")

# -- Load CSV ------------------------------------------------------------
csvfile = sys.argv[1] if len(sys.argv) > 1 else "timing_results.csv"

# data[study_type][size_label][nprocs] = [t1, t2, ...]
data = defaultdict(lambda: defaultdict(lambda: defaultdict(list)))

with open(csvfile) as f:
    reader = csv.DictReader(f)
    for row in reader:
        study  = row["study_type"]
        label  = f"{row['width']}x{row['height']}"
        nprocs = int(row["nprocs"])
        t      = float(row["wall_time_s"])
        data[study][label][nprocs].append(t)

def median(vals):
    return statistics.median(vals)

# -- Strong Scaling ------------------------------------------------------
print("\n" + "="*65)
print("  STRONG SCALING")
print("="*65)
print(f"  {'Size':<12} {'P':>4}  {'T(P) s':>10}  {'Speedup':>8}  {'Efficiency
```

```
    ':>10}")
print("-"*65)

ss_data = {}   # ss_data[label] = {nprocs: median_t}
for label, pdict in sorted(data["strong"].items()):
    sorted_p = sorted(pdict)
    t_base   = median(pdict[sorted_p[0]])  # smallest P is the baseline
    ss_data[label] = {}
    for p in sorted_p:
        t = median(pdict[p])
        speedup = t_base / t
        nworkers = p - 1  # rank 0 is master-only
        efficiency = speedup / nworkers if nworkers > 0 else float("nan")
        ss_data[label][p] = t
        print(f"  {label:<12} {p:>4}  {t:>10.4f}  {speedup:>8.3f}  {efficiency
    :>10.3f}")
    print()

# -- Weak Scaling ------------------------------------------------------------
print("="*65)
print("  WEAK SCALING  (normalized time - ideal = 1.000)")
print("="*65)
print(f"  {'P':>4}  {'Size':<12}  {'T(P) s':>10}  {'Norm T':>8}")
print("-"*65)

ws_norm = {}
p_list_ws = []
t_list_ws = []

ws_all = data["weak"]
first_entry = None

for label, pdict in sorted(ws_all.items()):
    for p, vals in sorted(pdict.items()):
        t = median(vals)
        if first_entry is None:
            first_entry = t
        norm = t / first_entry
        ws_norm[p] = norm
        p_list_ws.append(p)
        t_list_ws.append(t)
        print(f"  {p:>4}  {label:<12}  {t:>10.4f}  {norm:>8.3f}")

print()

# -- Amdahl's Law Fit --------------------------------------------------------
if HAS_MPL and ss_data:
    import numpy as np

    print("="*65)
    print("  AMDAHL'S LAW FIT  S(P) = 1 / (f + (1-f)/P)")
    print("="*65)

    for label, pdict in ss_data.items():
        procs   = sorted(pdict)
        t_base  = pdict[procs[0]]
        speedups = [t_base / pdict[p] for p in procs]
        p_arr   = np.array([float(p) for p in procs])
        s_arr   = np.array(speedups)
```

```
114
115         # Least-squares fit: s ≈ 1/(f + (1-f)/p)  → linearize
116         # 1/s = f + (1-f)/p  →  1/s = (1 - 1/p)*f + 1/p
117         inv_s = 1.0 / s_arr
118         X = 1.0 - 1.0 / p_arr
119         f_fit = float(np.polyfit(X, inv_s - 1.0/p_arr, 0)[0])  # intercept only
    approx
120         # Better: direct least squares
121         A = np.column_stack([X, 1.0/p_arr])
122         result = np.linalg.lstsq(A, inv_s, rcond=None)
123         f_fit, _ = result[0]
124         f_fit = max(0.0, min(1.0, f_fit))
125         p_max = 1.0 / f_fit if f_fit > 0 else float("inf")
126         print(f"  {label:<12}  serial fraction f ≈ {f_fit:.4f}  "
127               f"→ theoretical max speedup ≈ {p_max:.1f}×")
128     print()
129
130 # -- Plots ------------------------------------------------------------------
131 if HAS_MPL:
132
133     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
134     fig.suptitle("MPI Mandelbrot - Scaling Study", fontsize=14, fontweight="bold")
135
136     # -- Strong scaling plot ------------------------------------------------
137     ax = axes[0]
138     colors = plt.cm.tab10.colors
139
140     for i, (label, pdict) in enumerate(sorted(ss_data.items())):
141         procs    = sorted(pdict)
142         t_base   = pdict[procs[0]]
143         speedups = [t_base / pdict[p] for p in procs]
144         ax.plot(procs, speedups, "o-", color=colors[i % 10], label=label,
    linewidth=2)
145
146     # Ideal (linear) speedup line
147     all_p = sorted({p for pd in ss_data.values() for p in pd})
148     workers = [p - 1 for p in all_p]
149     ax.plot(all_p, workers, "k--", linewidth=1, alpha=0.5, label="Ideal (P-1)")
150
151     ax.set_xlabel("Number of MPI Processes (P)")
152     ax.set_ylabel("Speedup S(P)")
153     ax.set_title("Strong Scaling - Speedup")
154     ax.legend()
155     ax.grid(True, alpha=0.3)
156
157     # -- Weak scaling plot --------------------------------------------------
158     ax2 = axes[1]
159     ws_p = sorted(ws_norm)
160     ws_t = [ws_norm[p] for p in ws_p]
161
162     ax2.plot(ws_p, ws_t, "s-", color="steelblue", linewidth=2, label="Measured")
163     ax2.axhline(1.0, color="k", linestyle="--", linewidth=1, alpha=0.5, label="
    Ideal")
164
165     ax2.set_xlabel("Number of MPI Processes (P)")
166     ax2.set_ylabel("Normalized Wall Time  T(P) / T(2)")
167     ax2.set_title("Weak Scaling - Communication Overhead")
168     ax2.legend()
169     ax2.grid(True, alpha=0.3)
```

```
170
171     plt.tight_layout()
172     plt.savefig("scaling_results.png", dpi=150, bbox_inches="tight")
173     print("Plot saved to: scaling_results.png")
174     plt.show()
```

E Raw Job Output: mandelbrot_94.out

```
Launch Time: Sun Apr 26 01:28:38 PM EDT 2026
Running scaling study across Three Idiots cluster...
=== Strong Scaling Study ===
  Problem: 1024x1024, max_iter=1000
    P=2 trial=1 → 0.841948s
    P=2 trial=2 → 0.843172s
    P=2 trial=3 → 0.841980s
    P=3 trial=1 → 0.426557s
    P=3 trial=2 → 0.424308s
    P=3 trial=3 → 0.431425s
    P=4 trial=1 → 0.217534s
    P=4 trial=2 → 0.217741s
    P=4 trial=3 → 0.216721s
    P=5 trial=1 → 0.180834s
    P=5 trial=2 → 0.181978s
    P=5 trial=3 → 0.181990s
    P=6 trial=1 → 0.141886s
    P=6 trial=2 → 0.151726s
    P=6 trial=3 → 0.149512s
    P=7 trial=1 → 0.113608s
    P=7 trial=2 → 0.117481s
    P=7 trial=3 → 0.118577s
    P=8 trial=1 → 0.103302s
    P=8 trial=2 → 0.104725s
    P=8 trial=3 → 0.103900s
    P=9 trial=1 → 0.093453s
    P=9 trial=2 → 0.094972s
    P=9 trial=3 → 0.092812s
    P=10 trial=1 → 0.080125s
    P=10 trial=2 → 0.079559s
    P=10 trial=3 → 0.079747s
    P=11 trial=1 → 0.073551s
    P=11 trial=2 → 0.074182s
    P=11 trial=3 → 0.074350s
    P=12 trial=1 → 0.069743s
    P=12 trial=2 → 0.069406s
    P=12 trial=3 → 0.068098s
    P=13 trial=1 → 0.061333s
    P=13 trial=2 → 0.062152s
    P=13 trial=3 → 0.061848s
    P=14 trial=1 → 0.059407s
    P=14 trial=2 → 0.060158s
    P=14 trial=3 → 0.058940s
    P=15 trial=1 → 0.057164s
    P=15 trial=2 → 0.056242s
    P=15 trial=3 → 0.055330s
    P=16 trial=1 → 0.052480s
    P=16 trial=2 → 0.051437s
    P=16 trial=3 → 0.051663s
    P=17 trial=1 → 0.049906s
    P=17 trial=2 → 0.050107s
    P=17 trial=3 → 0.050692s
    P=18 trial=1 → 0.048552s
    P=18 trial=2 → 0.047981s
    P=18 trial=3 → 0.049166s
    P=19 trial=1 → 0.046024s
```

```
    P=19 trial=2 → 0.045217s
    P=19 trial=3 → 0.045177s
    P=20 trial=1 → 0.045567s
    P=20 trial=2 → 0.045821s
    P=20 trial=3 → 0.044422s
    P=21 trial=1 → 0.044695s
    P=21 trial=2 → 0.044071s
    P=21 trial=3 → 0.042834s
    P=22 trial=1 → 0.041417s
    P=22 trial=2 → 0.041154s
    P=22 trial=3 → 0.041635s
    P=23 trial=1 → 0.040834s
    P=23 trial=2 → 0.041065s
    P=23 trial=3 → 0.042357s
    P=24 trial=1 → 0.039943s
    P=24 trial=2 → 0.041106s
    P=24 trial=3 → 0.040791s
  Problem: 2048x2048, max_iter=1000
    P=2 trial=1 → 2.883590s
    P=2 trial=2 → 2.886955s
    P=2 trial=3 → 2.891694s
    P=3 trial=1 → 1.521275s
    P=3 trial=2 → 1.471966s
    P=3 trial=3 → 1.519498s
    P=4 trial=1 → 0.830713s
    P=4 trial=2 → 0.830153s
    P=4 trial=3 → 0.831131s
    P=5 trial=1 → 0.637809s
    P=5 trial=2 → 0.637854s
    P=5 trial=3 → 0.639868s
    P=6 trial=1 → 0.525301s
    P=6 trial=2 → 0.527099s
    P=6 trial=3 → 0.531499s
    P=7 trial=1 → 0.420256s
    P=7 trial=2 → 0.431637s
    P=7 trial=3 → 0.426043s
    P=8 trial=1 → 0.367885s
    P=8 trial=2 → 0.371492s
    P=8 trial=3 → 0.367084s
    P=9 trial=1 → 0.330852s
    P=9 trial=2 → 0.339903s
    P=9 trial=3 → 0.338624s
    P=10 trial=1 → 0.283668s
    P=10 trial=2 → 0.290646s
    P=10 trial=3 → 0.295411s
    P=11 trial=1 → 0.270534s
    P=11 trial=2 → 0.281002s
    P=11 trial=3 → 0.265686s
    P=12 trial=1 → 0.247088s
    P=12 trial=2 → 0.246624s
    P=12 trial=3 → 0.247013s
    P=13 trial=1 → 0.221590s
    P=13 trial=2 → 0.224939s
    P=13 trial=3 → 0.224862s
    P=14 trial=1 → 0.208979s
    P=14 trial=2 → 0.211715s
    P=14 trial=3 → 0.209759s
    P=15 trial=1 → 0.204137s
    P=15 trial=2 → 0.197633s
```

```
    P=15 trial=3 → 0.197829s
    P=16 trial=1 → 0.180996s
    P=16 trial=2 → 0.189022s
    P=16 trial=3 → 0.184741s
    P=17 trial=1 → 0.178736s
    P=17 trial=2 → 0.178807s
    P=17 trial=3 → 0.176470s
    P=18 trial=1 → 0.172132s
    P=18 trial=2 → 0.170385s
    P=18 trial=3 → 0.168249s
    P=19 trial=1 → 0.157463s
    P=19 trial=2 → 0.160014s
    P=19 trial=3 → 0.157505s
    P=20 trial=1 → 0.151526s
    P=20 trial=2 → 0.154403s
    P=20 trial=3 → 0.160091s
    P=21 trial=1 → 0.147341s
    P=21 trial=2 → 0.149380s
    P=21 trial=3 → 0.148317s
    P=22 trial=1 → 0.138680s
    P=22 trial=2 → 0.138501s
    P=22 trial=3 → 0.140037s
    P=23 trial=1 → 0.138392s
    P=23 trial=2 → 0.136306s
    P=23 trial=3 → 0.137535s
    P=24 trial=1 → 0.138668s
    P=24 trial=2 → 0.137175s
    P=24 trial=3 → 0.133765s
  Problem: 4096x4096, max_iter=1000
    P=2 trial=1 → 10.266579s
    P=2 trial=2 → 10.267361s
    P=2 trial=3 → 10.266761s
    P=3 trial=1 → 5.145649s
    P=3 trial=2 → 5.148036s
    P=3 trial=3 → 5.144898s
    P=4 trial=1 → 3.082643s
    P=4 trial=2 → 3.081752s
    P=4 trial=3 → 3.077000s
    P=5 trial=1 → 2.372382s
    P=5 trial=2 → 2.362451s
    P=5 trial=3 → 2.368469s
    P=6 trial=1 → 1.949945s
    P=6 trial=2 → 1.967734s
    P=6 trial=3 → 1.948321s
    P=7 trial=1 → 1.592205s
    P=7 trial=2 → 1.583753s
    P=7 trial=3 → 1.584629s
    P=8 trial=1 → 1.394240s
    P=8 trial=2 → 1.395647s
    P=8 trial=3 → 1.430385s
    P=9 trial=1 → 1.253888s
    P=9 trial=2 → 1.256574s
    P=9 trial=3 → 1.241682s
    P=10 trial=1 → 1.109210s
    P=10 trial=2 → 1.100834s
    P=10 trial=3 → 1.111930s
    P=11 trial=1 → 1.021461s
    P=11 trial=2 → 1.008528s
    P=11 trial=3 → 1.005199s
```

```
    P=12 trial=1 → 0.941793s
    P=12 trial=2 → 0.958283s
    P=12 trial=3 → 0.947971s
    P=13 trial=1 → 0.857405s
    P=13 trial=2 → 0.864970s
    P=13 trial=3 → 0.876483s
    P=14 trial=1 → 0.825039s
    P=14 trial=2 → 0.823426s
    P=14 trial=3 → 0.824870s
    P=15 trial=1 → 0.788748s
    P=15 trial=2 → 0.783621s
    P=15 trial=3 → 0.786954s
    P=16 trial=1 → 0.725021s
    P=16 trial=2 → 0.741966s
    P=16 trial=3 → 0.730220s
    P=17 trial=1 → 0.697247s
    P=17 trial=2 → 0.695693s
    P=17 trial=3 → 0.689010s
    P=18 trial=1 → 0.657662s
    P=18 trial=2 → 0.660748s
    P=18 trial=3 → 0.664968s
    P=19 trial=1 → 0.634975s
    P=19 trial=2 → 0.634194s
    P=19 trial=3 → 0.627041s
    P=20 trial=1 → 0.601462s
    P=20 trial=2 → 0.608260s
    P=20 trial=3 → 0.605797s
    P=21 trial=1 → 0.576046s
    P=21 trial=2 → 0.584487s
    P=21 trial=3 → 0.583090s
    P=22 trial=1 → 0.559837s
    P=22 trial=2 → 0.553645s
    P=22 trial=3 → 0.542940s
    P=23 trial=1 → 0.536119s
    P=23 trial=2 → 0.543631s
    P=23 trial=3 → 0.526352s
    P=24 trial=1 → 0.522558s
    P=24 trial=2 → 0.526497s
    P=24 trial=3 → 0.519375s

=== Weak Scaling Study ===
  Base tile: 512x512, max_iter=1000
  P=2 size=512x512 trial=1 → 0.280408s
  P=2 size=512x512 trial=2 → 0.280114s
  P=2 size=512x512 trial=3 → 0.280123s
  P=3 size=512x512 trial=1 → 0.143112s
  P=3 size=512x512 trial=2 → 0.142551s
  P=3 size=512x512 trial=3 → 0.144377s
  P=4 size=1024x1024 trial=1 → 0.216660s
  P=4 size=1024x1024 trial=2 → 0.216639s
  P=4 size=1024x1024 trial=3 → 0.215165s
  P=5 size=1024x1024 trial=1 → 0.182143s
  P=5 size=1024x1024 trial=2 → 0.182673s
  P=5 size=1024x1024 trial=3 → 0.183074s
  P=6 size=1024x1024 trial=1 → 0.152438s
  P=6 size=1024x1024 trial=2 → 0.152085s
  P=6 size=1024x1024 trial=3 → 0.151955s
  P=7 size=1024x1024 trial=1 → 0.117498s
  P=7 size=1024x1024 trial=2 → 0.117437s
```

```
  P=7 size=1024x1024 trial=3 → 0.117355s
  P=8 size=1024x1024 trial=1 → 0.103090s
  P=8 size=1024x1024 trial=2 → 0.102959s
  P=8 size=1024x1024 trial=3 → 0.102264s
  P=9 size=1536x1536 trial=1 → 0.196963s
  P=9 size=1536x1536 trial=2 → 0.202756s
  P=9 size=1536x1536 trial=3 → 0.196484s
  P=10 size=1536x1536 trial=1 → 0.168238s
  P=10 size=1536x1536 trial=2 → 0.171340s
  P=10 size=1536x1536 trial=3 → 0.175276s
  P=11 size=1536x1536 trial=1 → 0.157369s
  P=11 size=1536x1536 trial=2 → 0.158992s
  P=11 size=1536x1536 trial=3 → 0.155740s
  P=12 size=1536x1536 trial=1 → 0.147873s
  P=12 size=1536x1536 trial=2 → 0.146664s
  P=12 size=1536x1536 trial=3 → 0.146614s
  P=13 size=1536x1536 trial=1 → 0.134632s
  P=13 size=1536x1536 trial=2 → 0.130422s
  P=13 size=1536x1536 trial=3 → 0.134491s
  P=14 size=1536x1536 trial=1 → 0.125358s
  P=14 size=1536x1536 trial=2 → 0.130550s
  P=14 size=1536x1536 trial=3 → 0.124744s
  P=15 size=1536x1536 trial=1 → 0.119006s
  P=15 size=1536x1536 trial=2 → 0.116871s
  P=15 size=1536x1536 trial=3 → 0.120919s
  P=16 size=2048x2048 trial=1 → 0.183706s
  P=16 size=2048x2048 trial=2 → 0.182392s
  P=16 size=2048x2048 trial=3 → 0.181628s
  P=17 size=2048x2048 trial=1 → 0.179113s
  P=17 size=2048x2048 trial=2 → 0.176755s
  P=17 size=2048x2048 trial=3 → 0.175136s
  P=18 size=2048x2048 trial=1 → 0.169227s
  P=18 size=2048x2048 trial=2 → 0.168179s
  P=18 size=2048x2048 trial=3 → 0.169399s
  P=19 size=2048x2048 trial=1 → 0.158363s
  P=19 size=2048x2048 trial=2 → 0.156951s
  P=19 size=2048x2048 trial=3 → 0.157767s
  P=20 size=2048x2048 trial=1 → 0.152174s
  P=20 size=2048x2048 trial=2 → 0.151383s
  P=20 size=2048x2048 trial=3 → 0.152965s
  P=21 size=2048x2048 trial=1 → 0.147388s
  P=21 size=2048x2048 trial=2 → 0.147905s
  P=21 size=2048x2048 trial=3 → 0.147447s
  P=22 size=2048x2048 trial=1 → 0.140829s
  P=22 size=2048x2048 trial=2 → 0.141577s
  P=22 size=2048x2048 trial=3 → 0.140519s
  P=23 size=2048x2048 trial=1 → 0.137474s
  P=23 size=2048x2048 trial=2 → 0.134758s
  P=23 size=2048x2048 trial=3 → 0.136123s
  P=24 size=2048x2048 trial=1 → 0.132514s
  P=24 size=2048x2048 trial=2 → 0.136338s
  P=24 size=2048x2048 trial=3 → 0.136001s

Results written to: timing_results.csv
Run 'python3 analyze.py' to compute speedup, efficiency, and generate plots.
Job finished at: Sun Apr 26 01:34:25 PM EDT 2026
```